NEA - Group Creator for School Projects

Amelie Machin

Contents

1 Analysis **2**
1.1 Problem statement . . . 2
1.2 Research . . . 2
1.2.1 Sources of inspiration . . . 2
1.2.2 Algorithm . . . 3
1.2.3 Inputting data . . . 4
1.2.4 Accounts . . . 4
1.2.5 Internal data storage . . . 4
1.2.6 External data storage . . . 5
1.2.7 GUI library . . . 5
1.3 Dialogue . . . 5
1.4 Objectives . . . 7
1.5 Modelling . . . 12

2 Design **15**
2.1 GUI . . . 15
2.2 Databases . . . 22
2.3 Inputs and outputs . . . 22
2.4 Algorithm . . . 24
2.4.1 Pseudocode . . . 24
2.4.2 Fitness algorithm . . . 26
2.4.3 Diagrams . . . 26

3 Technical solution **28**
3.1 main.py . . . 28
3.2 algoClasses.py . . . 44
3.3 geneticAlgo.py . . . 45

4 Testing **46**

5 Evaluation **50**

1 Analysis

1.1 Problem statement

To form the perfect grouping for a group project within a school, many factors need to be taken into consideration to prevent students from feeling incapable of completing the project. These factors include, but are not limited to, skills of the students, skills required by the project, and number of students in the group.

This project aims to create a tool to aid teachers in ensuring the students are separated into groups where the students are best able to get along and combine their individual skills to achieve their project's aims. The user can input the data that they want to be taken into consideration by the program and it will output a set of groups. This is in the form of a table on a GUI - to present the results in a readable way - and an option to export the data.

This program will not find, assume, or generate information about the students or project, it will only work with the data inputted to assign a group of students to a project.

Teachers will benefit from this tool as they don't have to spend as much time or effort creating groups for the projects they set for their students, so they can spend their time and effort on helping and teaching the students instead. Also, when the students are working on their projects, they are more likely to be focused on their work, so require less oversight from the teacher.

Students will also benefit from this tool as they would be able to apply their skills to a project in a group of people they can enjoy time with, rather than being stuck on a project they have no interest in with people they do not get along with. This means they are more likely to enjoy their time at school, be able to get more work done, and therefore be able to learn more, as opposed to being unmotivated and distracted in a poorly made group.

1.2 Research

1.2.1 Sources of inspiration

1. Random group generator from ClassTools.net [Fig. 1]
   https://www.classtools.net/random-group-generator/
   - This tool is used frequently by my teachers to create random groups for group activities.
   - The GUI is very simple to use as there is little data that needs to be entered, so can all fit down the left of the screen.
   - The output is very easy to read as each group is visually separated using both coloured boxes and numbering. [Obj. 3b]
   - However, this tool is too simple because it lacks any way to group with conditions, which is part of my scope, as they are purely randomised. This means that teams could quit often be unfairly balanced in terms of skill, or group students together that are known

to not work well together. The teacher's only option to solve this is to re-roll until the output is suitable enough for the task manually. This gave me the idea for the algorithm to be the simulation of this re-rolling and selection of optimal outputs.

2. keamk [Fig. 2] [Fig. 3]
   https://www.keamk.com/random-team-generator

   - This tool creates groups that all have a balanced level of skill.
   - I like how the skill level is inputted as a rating between 1-5 as this is intuitive for the user.
   - I also like the extra details that are outputted, such as how many are in each group next to the group name and the skill levels next to each individual. [Obj. 3a]
   - Furthermore, the option to go back to make small changes to the data inputted and get new groupings was very quick and useful. [Obj. 1g]
   - However, this tool can only create balanced groups, not groups with different objectives as is in my scope, and there is no way to assign a certain number of participants to each group, as the program puts the same number (or one away) of participants in each group. [Obj. 2c-d]

These tools influenced me to reduce the complexity of my GUI, and gave me ideas such as to keep groups visually separate using colour as well as tables, adding the option to go back and edit the inputted data later, and some summary statistics to include with the outputs.

1.2.2 Algorithm

I have researched into representing students and projects into bipartite graphs, and then using matching algorithms such as the alternate path algorithm to join students to projects [Fig. 4]. However, this will only match one student to one project, not groups so is not suitable. I then researched applying the algorithm repeatedly to graphs with randomised students, but this means that some students will never be joined into a group purely based on chance and that the groups formed are such that each individual is best for the project, not that the overall group is best suited for the project, so teamwork will not as good as it could be. I then researched using a grouping algorithm, such as https://help.feedbackfruits.com/hc/en-us/articles/23527084211346-Group-Formation-Algorithm-Explainer to create similar groups then using bipartite matching algorithms to match them. However this only creates groups with similar skill sets, not skill sets that vary based on the project.

I decided on a genetic algorithm where each individual is made up of a possible solution and the fitness score is an average between how suited each group is to its assigned project [Fig. 5]. The individuals with higher fitness scores are discarded and the lower ones are randomly selected to crossover to form new

individuals. Some of these individuals formed may contain error such as the same student multiple times, or a missing student, so that must be corrected before continuing. Then, random mutations are applied to some individuals, which may involve swapping of students between groups, moving some students to an optional project, or moving some students to group that isn't full. New fitness scores are calculated and the process repeats a predetermined number of times [Fig. 9]. I chose this because genetic algorithms are very adaptable to varying conditions and constraints, such as the ones that the user chooses in this program. This will allow the program to test different combinations of projects and students, test different group sizes for a project, and swap in and out optional projects to find better and better solutions until it reaches a high-quality solution that fits all the user's needs. This fits the project scope far better than all previously researched algorithms.

1.2.3 Inputting data

For the inputting of data, I have decided that it can be inputted through txt and csv files [Obj. 1a], manually on the GUI [Obj. 1b], or any combination of those methods [Obj. 1c]. This is because the data for different individuals and different projects could be stored in different ways within the same system, and manual tweaks may be needed by the user to ensure all data is as is needed by the application. I will also include the option to import the data from an Excel spreadsheet at the request of my third party [Obj. 1b].

1.2.4 Accounts

I have decided against implementing an account system because I don't think it will provide any value to the program. Accounts are typically used for ensuring data can only be accessed by the users with the correct permissions. As this program will be designed to be run locally, all the data will be stored and only accessible on the machine the program was run on, so the OS can handle user security.

Another reason for accounts is allow users to access their data that has been stored by a third party, especially within a cloud to allow for data to be quickly transferred between devices. I will implement a manual solution to this by allowing users to export and import all the data the program stores and the user can handle that data transfer themselves.

1.2.5 Internal data storage

I chose to store the data about the students and projects in their own classes, that are aggregated within an 'individual' class. The student and project classes contain their names, skills and the project will have the recommended group size. The individual will contain a project object, student objects, and current fitness score as well as methods for calculating fitness scores, crossover with other individuals, and mutations. [Fig. 7]

This keeps the attributes and methods stored together in the program to make the code easier to read and understand by a programmer.

1.2.6 External data storage

[Obj. 4] I have researched into whether information about projects and students should be stored somewhere, and if so, how. I have concluded that they should be stored in relational databases that are accessible and modifiable by entering the details into the graphical interface, which are passed as parameters into SQL statements. These can then be loaded as a list of checkboxes that can be selected and deselected by the user for that run. [Obj. 1c]

I chose to store the data because being able to make minor changes to the data and then re-running the program quickly is a very valuable feature that could save users' time and energy as they would not have to go through the entire process of entering every piece of data just for small changes to some of the data. This feature is present in keamk's grouping tool, and I found it very useful in my testing.

I chose databases to store the data because I have more experience working with relational databases and SQL then any other form of data storage, such as in files. The data is structured so databases would allow for efficient access and modification of specific entries.

1.2.7 GUI library

Between the two main python GUI libraries, pygame and Tkinter, I chose Tkinter because of the larger library of classes of pre-made, customisable widgets that can be used more easily to produce a simple, easy-to-use tool, such as a dialog that allows users to graphically navigate through their files to find their data file [Fig.6].

On the other hand, with my prior experience with pygame, I have found it to be more suited towards building games and graphically-complex tools, which is not what I am aiming for in this project. I am aiming to have a GUI that is quick and simple to create and edit, and has user-friendly features for inputting, editing, and reading data. I also have more experience with Tkinter, which will reduce the amount of time I spend on the GUI and allow me to focus on developing and improving the grouping algorithm and user experience.

1.3 Dialogue

My third party is Dr Ed Horncastle, the lead teacher for the Exeter Mathematics Certificate (EMC) at my school, which is where groups of 3 or 4 students work together on research projects set by University academics or companies. He is suitable because he has to make these groups based on the skills of the students and match them to projects every year, so he has a thorough understanding of how this tool could work and would benefit from using this tool. I interviewed him and asked him the following questions:

1. What is the current process for creating EMC groups?
    - All the metrics to consider are gathered into a spreadsheet and then groups are formed by hand.
    - Some projects don't end up getting groups assigned, especially those that have been set by a company that already has another project with an assigned group.
2. What are these metrics?
    - Students' own choice
    - There must be a mix of genders
    - The projects must be relevant to the A-level studies of that student. Someone who does not take computer science shouldn't be given a computer science based project
    - Those that work hard and those that laze off more easily should be mixed together. If this is their first project, this is inferred from GCSE results, otherwise it is based off the results of previous projects.
    - Special educational needs
    - Timetabling restrictions
    - Students do not get put into groups with others they have worked on projects with before
3. How are the student choices measured?
    - For some of the projects, the students give their top 3 choices of projects. Starting with the least popular project, the groups are formed by trying to minimise the sum of the positions the student's ranked the project in.
4. How is the data about the students and projects currently stored?
    - Excel spreadsheet
5. How would you prefer to input the data?
    - Student choices from a Microsoft Form and all other data from a Excel spreadsheet
6. How much data and in what format do you think the data should be outputted?
    - In tables with the project name, student names and summaries of information about each group such as gender ratio, how many take computer science or physics, and how many have special education needs.
    - As a simplified register in alphabetical order

7. How useful would storing project/student details locally, that can be loaded when needed, be for you?

    - Very

8. Are there any other features that would be useful for this tool to include?

    - Some extra features that could be added is timetabling for what order the projects are presented in, and itinerary for group trips out of school.

After this, I decided to include Excel spreadsheets as an input method [Obj. 1b], include options for projects to not have groups assigned despite being a part of the algorithm [Obj. 1f], take genders into account [Obj. 2e], have summary statistics presented alongside the groups [Obj. 3a], have the option to view the groups without any extra information [Obj. 3b], store the data locally for re-use at another time [Obj. 4].

I decided not to include student choice, student personality, special education needs, timetabling restrictions, or preventing grouping the same students together across multiple as adding too many restrictions to the algorithm will make it take it longer to run to create a set of groups that best matches all of these restrictions. These are the restrictions that were lower on Dr Horncastle's priorities. I will also not include timetabling or scheduling as these deviate quite far from my project's scope.

1.4 Objectives

1. Data can be entered into the program from external sources
    (a) Data can be entered manually into the GUI
    (b) Data can be entered through txt, csv, and Excel spreadsheet files
    (c) Data can be loaded from stores from previous uses of the program
    (d) Data can be entered through a combination of other methods
    (e) Student data must contain a name, gender, and skills
        i. Name must only contain alphanumeric characters and certain whitelisted characters
    (f) Project data must contain a name, maximum group size, minimum group size, whether or not the project must have a group or not, and skill requirements
        i. Name must only contain alphanumeric characters and certain whitelisted characters
        ii. Maximum group size must be greater than 0
        iii. Minimum group size must be greater than or equal to 0
    (g) Data can be added, edited, and removed at any time during the program's runtime

(h) Data must be entered before the algorithm is run

2. Students must be grouped together and to a project

   (a) All students must be put into a group

   (b) All students must have at least one skill that will allow them to contribute to the project, when possible

   (c) Projects must not have more students assigned to it than the maximum limit inputted by the user

   (d) Projects must not have less students assigned to it than the minimum limit inputted by the user

   (e) Groups must have a mix of genders

3. The resulting groups can be viewed

   (a) This can be through coloured tables displayed on the GUI that contain the project name, student names, number of students, gender ratio, and skill range

   (b) This can be through a register that displays projects in alphabetical order, grouped with its students in alphabetical order, all coloured to separate from each other, and no other information.

   (c) Both above options must automatically be saved as ... files in the local file directory.

4. All data about projects and students must be stored

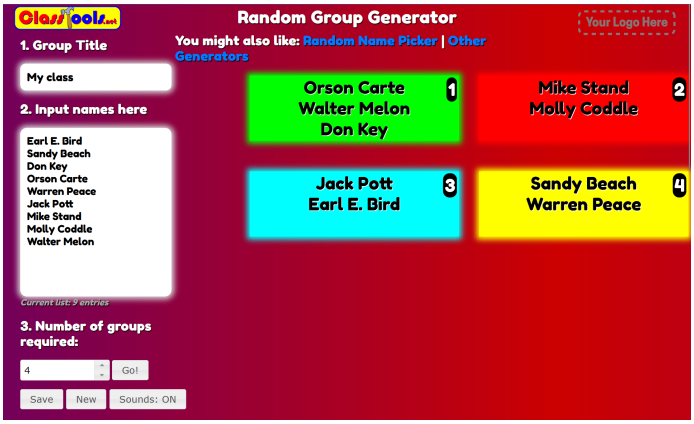


Figure 1: ClassTools's random group generator tool with inputs and outputs visible on the same page

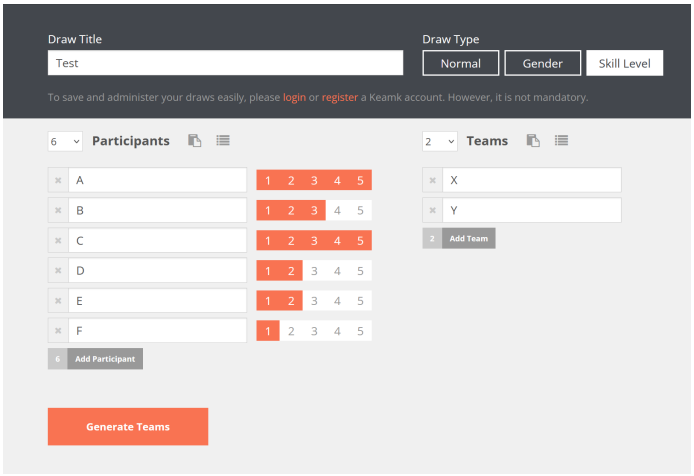


Figure 2: Keamk's menu for inputting data



Figure 3: Keamk's outputs

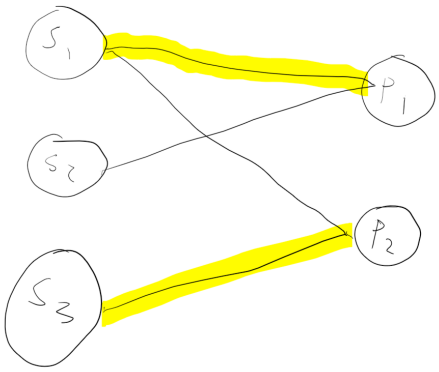


Figure 4: A bipartite graph with 3 student nodes on the left, 2 project nodes and the right, and some edges between them. A matching algorithm has been applied, with the matching highlighted in yellow. This has not met the scope because a student has not been matched.

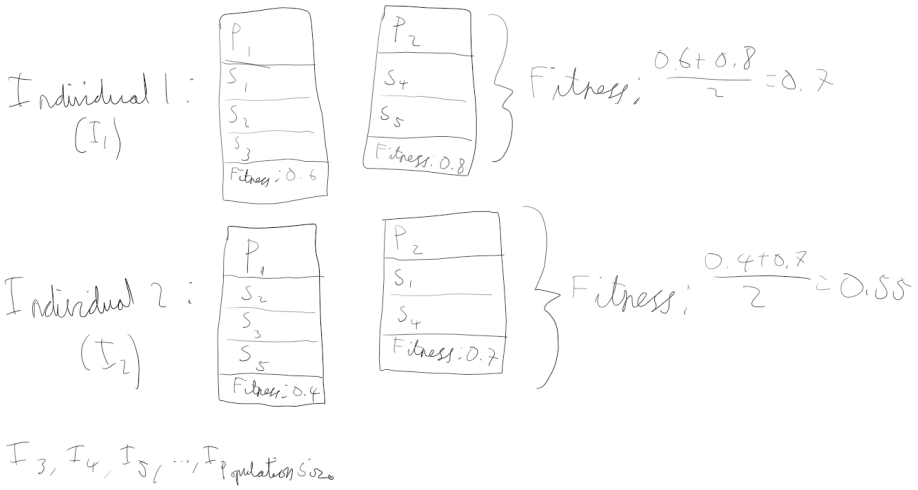


Figure 5: A sketch showing an example of a population, representing a set of possible solutions (individuals). Each individual represents a possible set of groups, and has a fitness score that represents how well every project's group matches the project's and user's requirements. This is calculated by averaging all the fitness scores of the groups it contains.

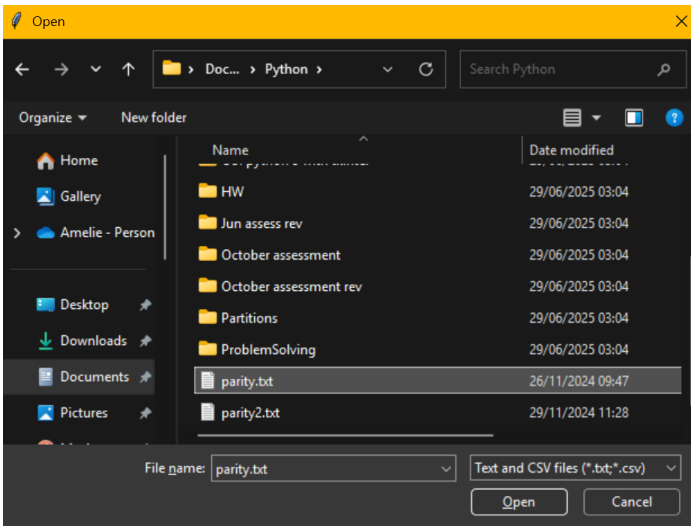


Figure 6: Tkinter's file dialog, configured to only show .txt and .csv files

1.5 Modelling

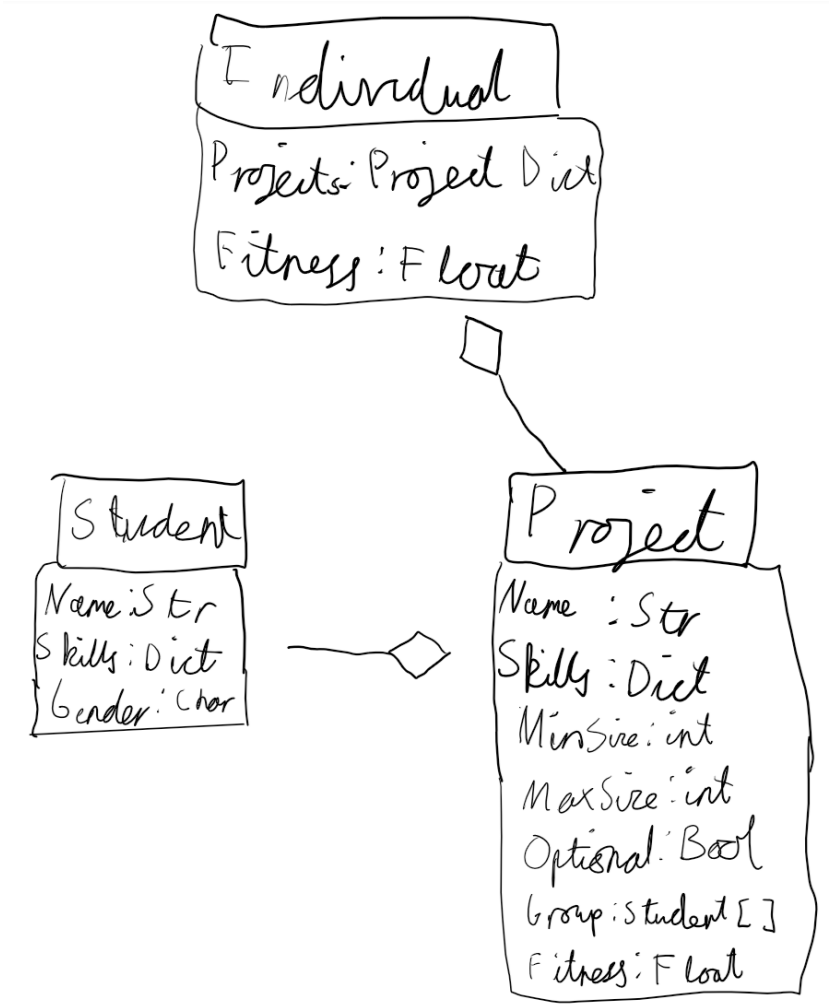


Figure 7: Class diagram showing relationships between the classes and their attributes used by the genetic algorithm.

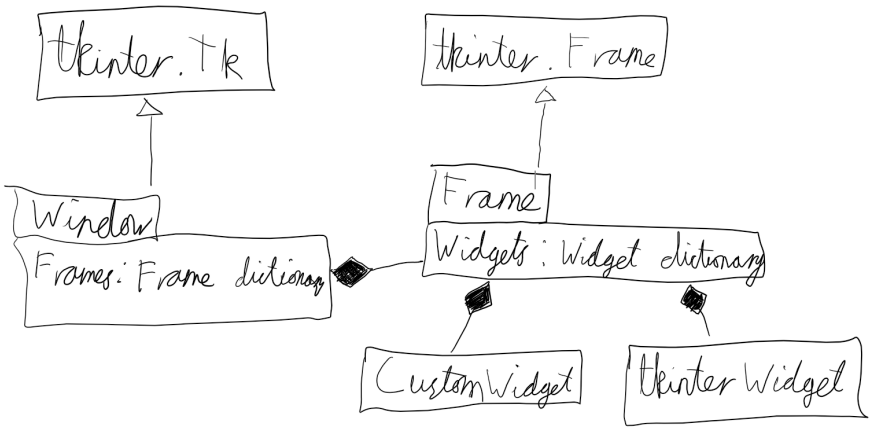


Figure 8: Class diagram showing relationships between the classes that form the GUI

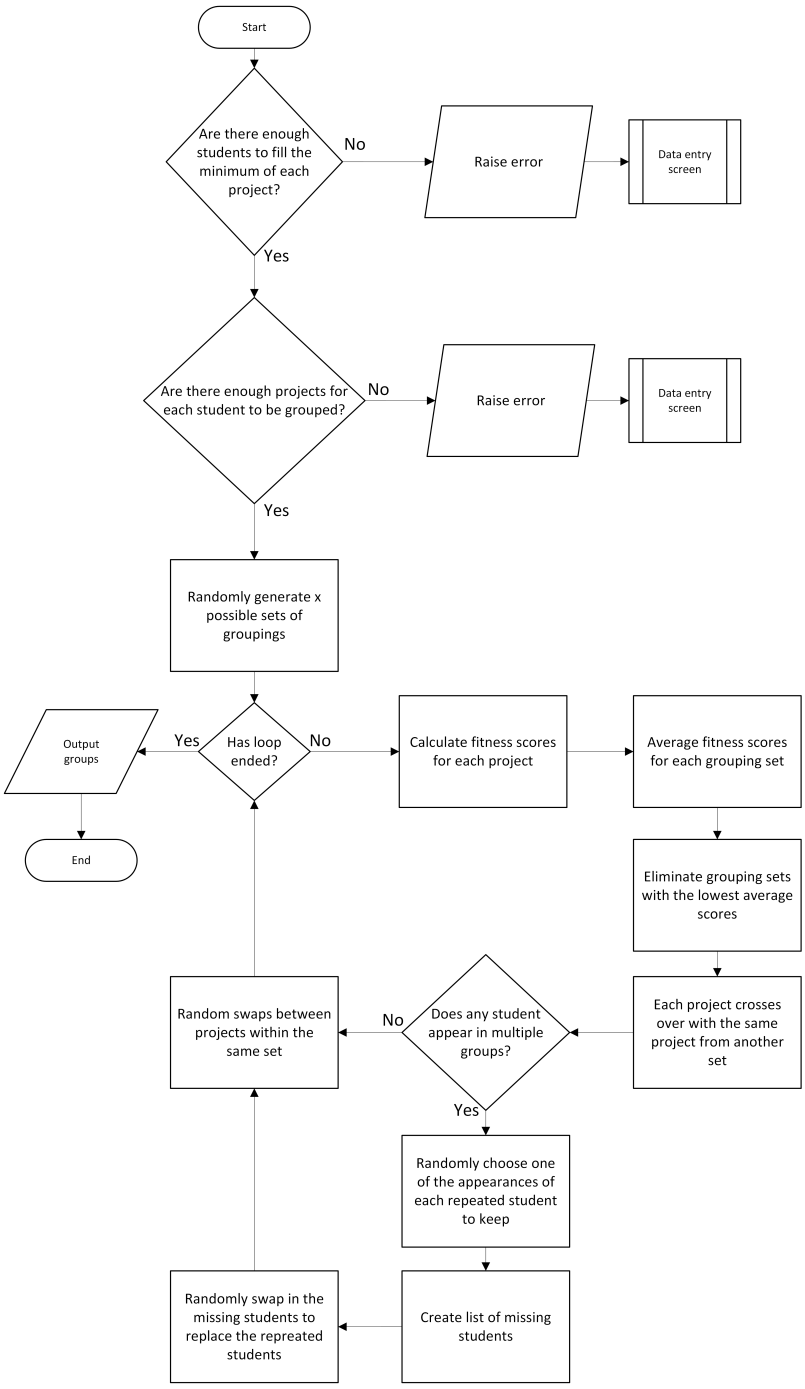


Figure 9: Flowchart showing the process for the genetic algorithm

2 Design

2.1 GUI

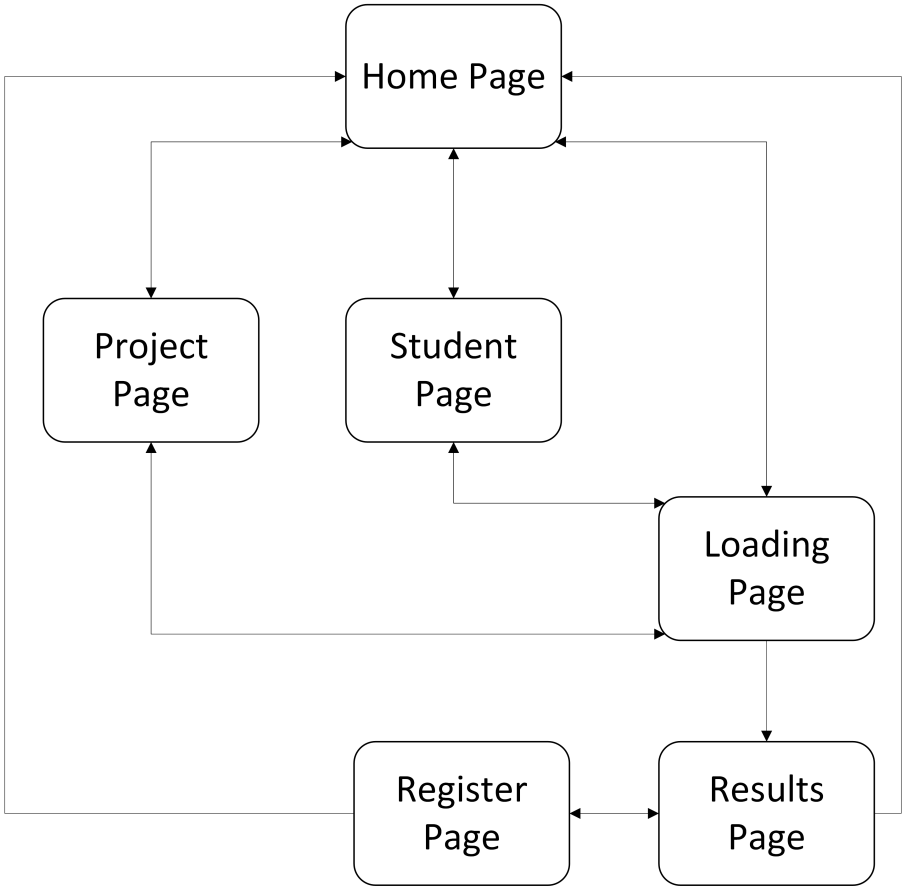


Figure 10: Page navigation diagram showing which pages can navigate to each other. Note: The student and project pages can only navigate to the previous page.

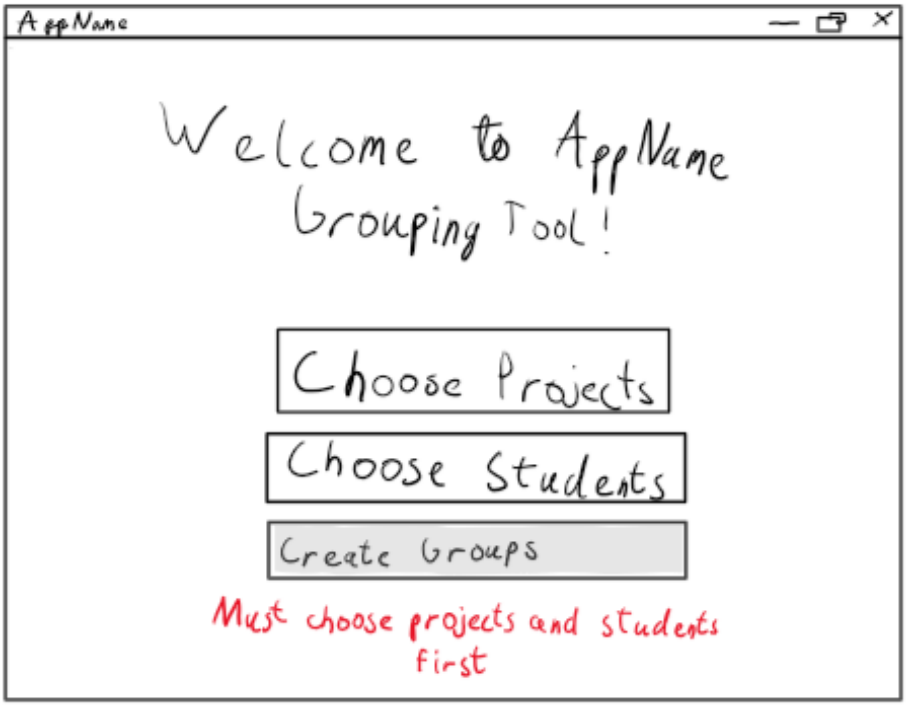


Figure 11: Sketch of the homepage layout that is the first page seen upon opening the app. The "Choose Projects" and "Choose Students" lead to the projects and students pages, respectively. The "Create Groups" button leads to the loading page and runs the grouping algorithm with the students and projects the user has selected. It is greyed out and not clickable unless at least one project and student has been selected [Obj. 1h].

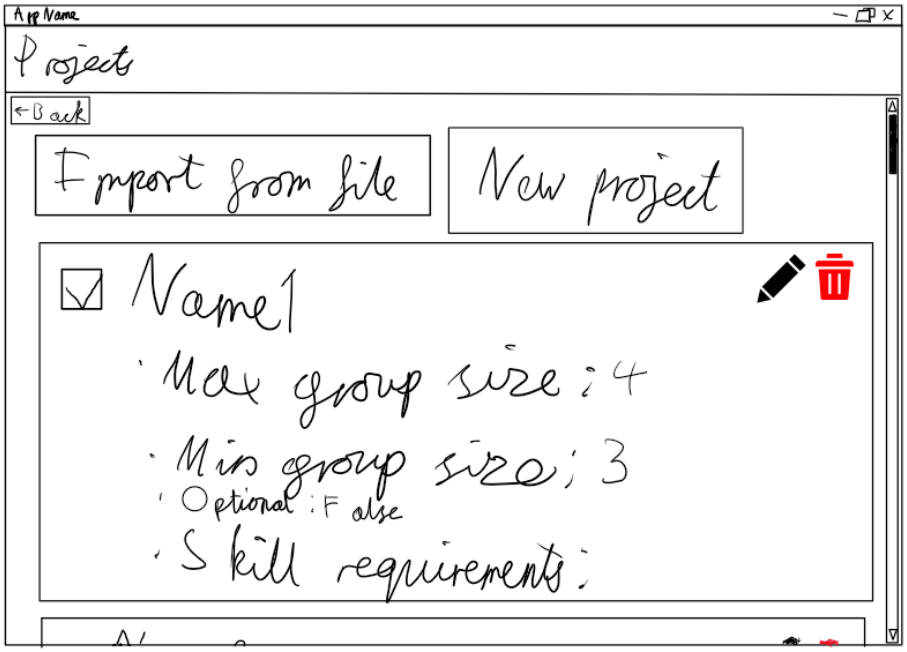


Figure 12: Sketch of the project page layout. The projects in the scrollable list are all those that are currently stored in the database [Obj. 1c]. There are options to import project data from supported files [Obj. 1b] and to manually enter project details at the top [Obj. 1a], which then get added to the database [Obj. 4] and the visual list [Obj. 1d]. The project details can also be edited or deleted from the database by pressing the buttons [Obj. 1g], which run parametrised SQL. Each project in the list features a checkbox, allowing users to select which of the projects in the database will be used in the grouping algorithm. Projects not selected will not have groups matched to them. This page also features a back button to go back to the home page.

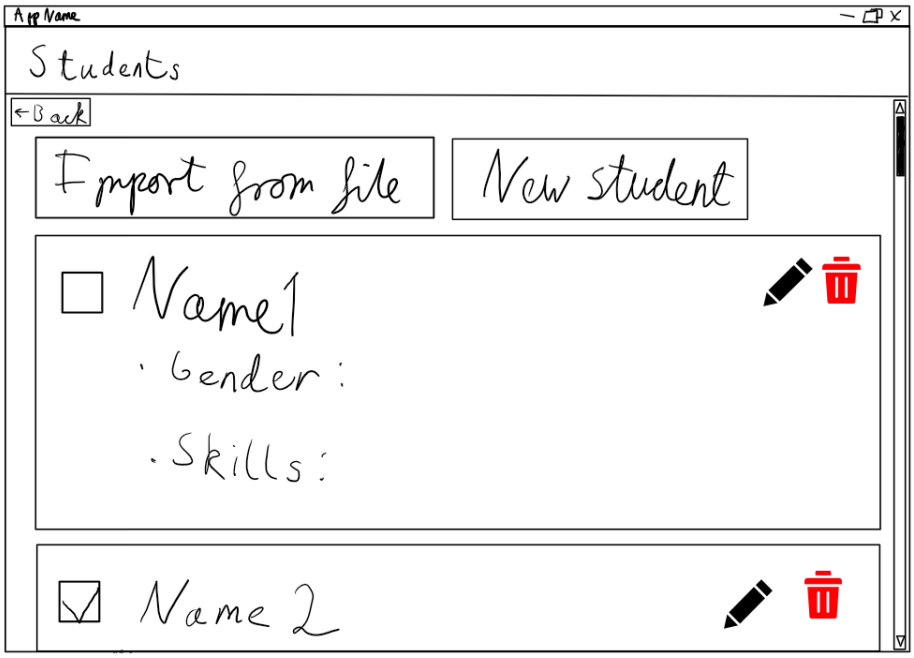


Figure 13: Sketch of the student page layout. The students in the scrollable list are all those that are currently stored in the database [Obj. 1c]. There are options to import student data from supported files [Obj. 1b] and to manually enter student details at the top [Obj. 1a], which then get added to the database [Obj. 4] and the visual list [Obj. 1d]. The student details can also be edited or deleted from the database by pressing the buttons [Obj. 1g], which run parametrised SQL. Each student in the list features a checkbox, allowing users to select which of the students in the database will be used in the grouping algorithm. Students not selected will not be put into groups. This page also features a back button to go back to the home page.

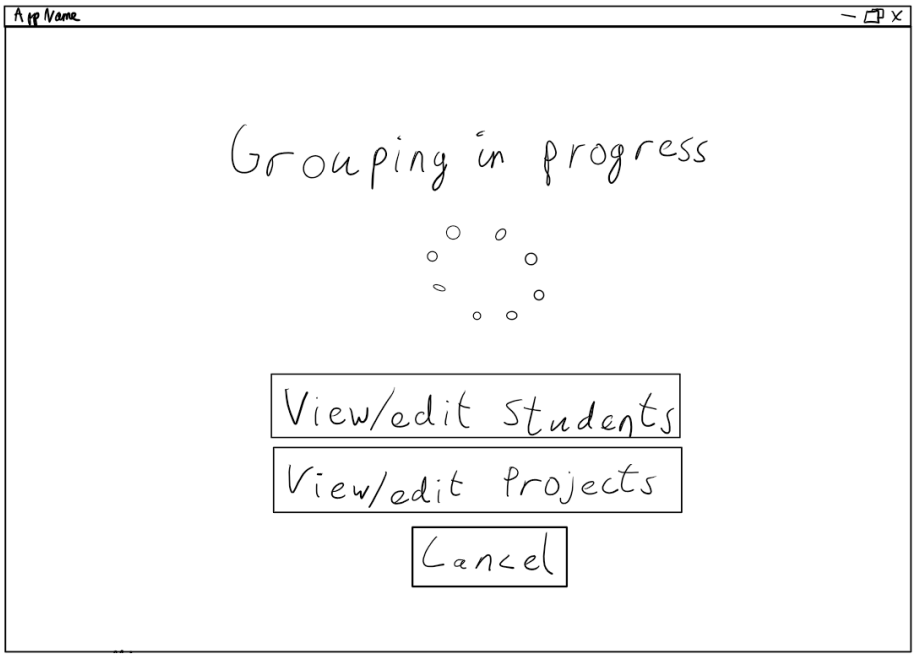


Figure 14: Sketch of the loading page layout. This page features a loading animation, and three buttons. The "View/edit Students" and "View/edit Projects" buttons lead to the Students and Projects pages, respectively [Obj. 1g]. The "Cancel" button ends the algorithm and leads to the home page. Once the algorithm has finished successfully, it moves onto the results page.

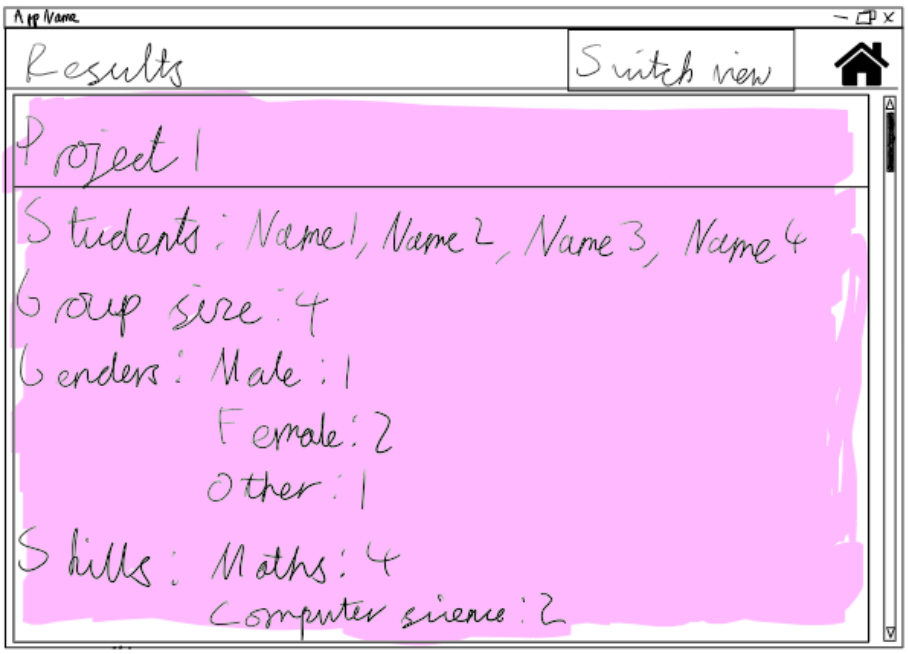


Figure 15: Sketch of the results page layout. It features a scrollable list of all the projects with the students grouped to it and summary statistics about the group, such as group size, gender ratio, and skills [Obj. 3a]. The "Switch view" leads to the register page and the home button leads to the home page.

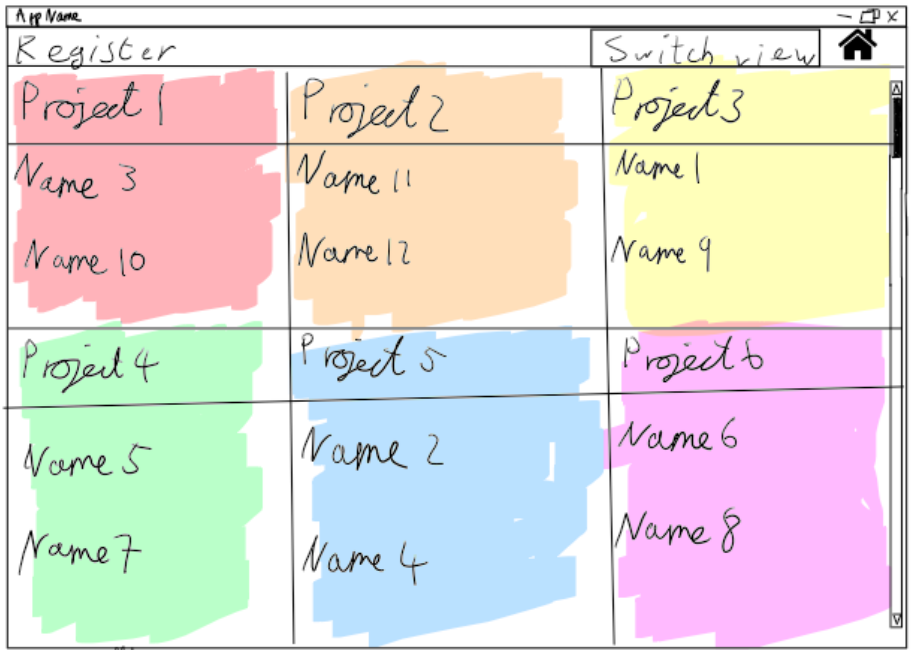


Figure 16: Sketch of the register page layout.

2.2 Databases

- Students(StudentID, Name, Gender)
- StudentSkills(StudentID, Skill)
- Projects(ProjectID, Name, MaxSize, MinSize, Optional)
- ProjectSkills(ProjectID, Skill)
- Results(ResultID, ProjectID, LocalTimeGenerated, EpochTimeGenerated)
- ResultStudentLink(ResultID, StudentID)

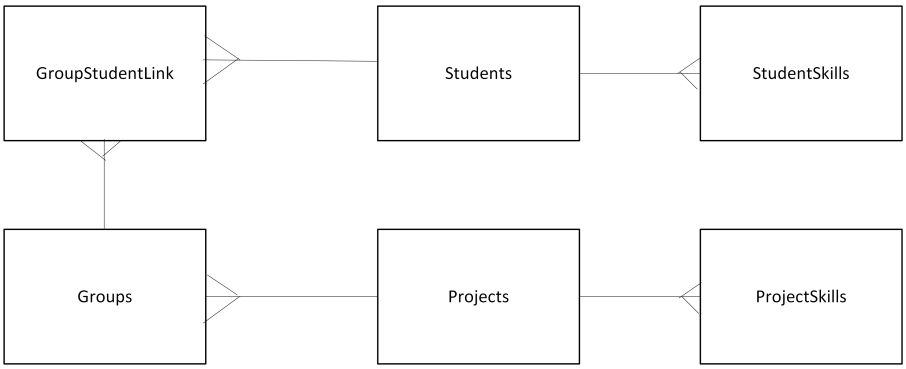


Figure 17: An entity relationship diagram showing the one-to-many relationships between the Students and Projects entities and the store of their skills to ensure all fields are atomic so the database is normalised.

2.3 Inputs and outputs

Table 1: An IPSO chart showing what data is inputted, how it is processed and stored, and how the results are outputted.

Input	Process	Storage	Output
Project details: - Name - Max group size - Min group size - Optional - Skill requirements	Validate input	Projects and ProjectSkills tables in database	Display on GUI [Fig. 12]

Student details: - Name - Gender - Skills	Validate input	Student and StudentSkills tables in database	Display on GUI [Fig. 13]
Settings: - Directory to store and load database - Target fitness for genetic algorithm - Max iterations for genetic algorithm - Size of populations for genetic algorithm	Validate input	JSON file	NA
"Create Groups" button on homepage pressed	Run genetic algorithm[List. 1]	Classes for Student, Project, and Individual [Fig. 7]	Scrollable list of projects and groups [Fig. 15]

Table 2: Validations and restrictions on data entry

Data	Data Type	Validation and Restrictions
Projects		
Name	String	- Required
Max group size	Integer	- Required - Positive integers only
Min group size	Integer	- Required - Positive integers only - Must be less than or equal to 'Max group size'
Optional	Boolean	- Lack of entry is equivalent to entry of false
Skill requirements	String	- Entries separated by commas
Students		
Name	String	- Required
Gender	String	
Skills	String	- Entries separated by commas
Settings		

Directory to store and load databases	String	- Default is the directory the program is stored in - Must be a valid directory - Program must have read-/write permissions in that directory
Target fitness for genetic algorithm	Float	- Default is 0.9 - Must be between 0 and 1 inclusive
Max iterations for genetic algorithm	Integer	- Default is 50 - Must be a positive integer not including 0
Size of populations for genetic algorithm	Integer	- Default is 100 - Must be a positive integer not including 0

2.4 Algorithm

2.4.1 Pseudocode

Listing 1: Pseudocode for the function that runs when on the loading page. It ensures it is possible to find groupings with the given requirements, then runs a genetic algorithm to find a good set of groupings.

```
1  TARGET_FITNESS = integer
2  MAX_ITERATIONS = integer
3  POPULATION_SIZE = integer
4
5
6  students = set of 'student' objects
7  projects = set of 'project' objects
8
9
10 #Ensure there aren't too many or too few students to meet
       ↪ requirements
11 minStudents = 0
12 maxStudents = 0
13
14 REPEAT for all projects:
15     maxStudents += project.maxSize
16     IF (not project.optional):
17         minStudents += project.minSize
18
19 #Check for too few students
20 IF (length of students < minStudents):
```

```
21      DisplayError("Error: Only {length of students} to
            ↪ fill a minimum of {minStudents} places.")
22      HomePage()
23
24  #Check for too many students
25  ELSE IF (length of students > maxStudents): #Checks for
        ↪ too many
26      DisplayError("{length of students} is too many to fit
            ↪  in {maxStudents} places.")
27      HomePage()
28
29
30  #Generate initial grouping sets and average fitness
        ↪ scores
31  population = list
32  REPEAT for i = 0 to POPULATION_SIZE - 1:
33      groups = GenInitialGroups(projects, students)
34      avgFit = (SUM fitness of each group)/(length of
            ↪ groups)
35      APPEND groups,avgFit to population
36
37  SORT population by avgFit
38
39  #Genetic algorithm
40  REPEAT for i = 1 to MAX_ITERATIONS:
41      #Ends early when one set of groupings is good enough
42      IF (highest avgFit > TARGET_FITNESS):
43          BREAK
44
45      #Eliminate the worst half of grouping sets
46      population = first (POPULATION_SIZE FLOORDIV 2) of
            ↪ population
47      new_pop = []
48
49      #Create next population
50      REPEAT for j = 1 to 90% of POPULATION_SIZE:
51          #Crossover random groupings
52          groups = Mate(random group from population,
                ↪ random group from population)
53
54          #Ensure all students appear once
55          valid = verifyGroupings(groups)
56          IF (not valid):
57              fixGroupings(groups)
58
```

```
59          #Random mutations to prevent convergence towards
                ↪ a local best solution
60          Mutate(groups)
61
62          #Find the average fitness score for each new
                ↪ grouping
63          avgFit = (SUM fitness of each group)/(length of
                ↪ groups)
64          APPEND groups,avgFit to new_pop
65
66      #Combine the crossover with 10% elitism
67      population = new_pop + top 10% of population
68      SORT population by avgFit
69
70
71  return group with highest avgFit
```

2.4.2 Fitness algorithm

Still working on it.

2.4.3 Diagrams

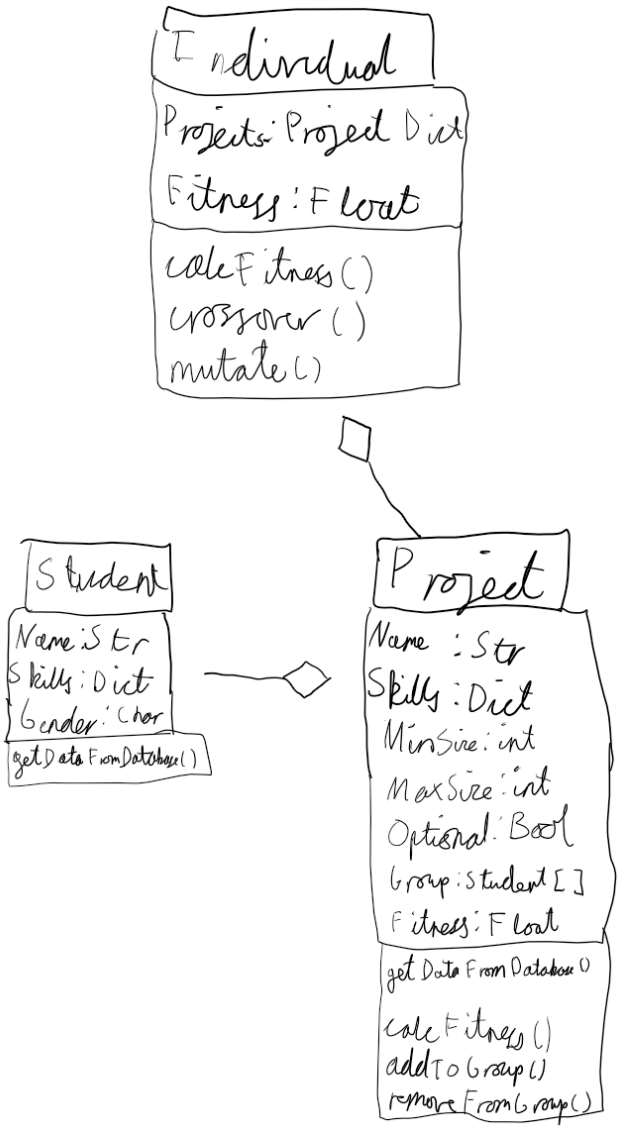


Figure 18: A class diagram showing the attributes and methods of the classes used by the genetic algorithm, and the relationships between each class.

3 Technical solution

Contents

3.1 main.py 28
3.2 algoClasses.py 44
3.3 geneticAlgo.py 45

main.py is the main file that is run by the user. It contains the GUI, and calls to the functions and classes in the other files when needed. All the pages outlined above [Fig. 10] have their own classes, whose objects are stored in the window class. The page classes' attributes are mostly widgets, both tkinter widgets and custom widgets (such as Header, ScrollableFrame, SelectionBox, and ProjectBox), or the attributes store the result of user interaction, such as the state of the checkboxes on the project page. The methods all code for the processing and outputting the result of human interaction with the GUI.

algoClasses.py contains the Individual, Student, and Project classes for use by the genetic algorithm. These follow the design of the above class diagram [Fig. 18].They are stored in a separate file for re-use across multiple files.

geneticAlgo.py contains the code for genetic algorithm that calculates the groups. it follows the design of the above pseudocode [Listing 1]

3.1 main.py

```
import tkinter as tk
from tkinter import ttk
from tkinter import filedialog
from tkinter import messagebox
import sqlite3 as sql

#=====GLOBAL CONSTANTS=====

BG_COLOUR="#FFFFFF"
HEADER_COLOUR="#FFDB32"
LARGE_FONT=("Arial",30)
DATABASE = "Code/database.db"

#=====ROOT WINDOW=====
#Constants for styling included within

class Window(tk.Tk):
    def __init__(self):
        #Setup window
        tk.Tk.__init__(self)
```

```
        self.state('zoomed')
        self.geometry(f"{self.winfo_screenwidth()-10}x{self.winfo_screenheight()}")
        self.title('Student Grouping Tool')
        self.config(background=BG_COLOUR)

        #=====STYLES=====

        HEADER_BUTTON_STYLE=ttk.Style()
        HEADER_BUTTON_STYLE.configure("Header.TButton",background=HEADER_COLOUR)

        #================
        #Dictionary that will store all loaded pages
        self.frames={}
        #Show the home page first
        self.show_frame(HomePage)

    #Method for changing currently open page
    def show_frame(self, f):
        #If the page is already loaded, show it
        if f in self.frames.keys():
            frame = self.frames[f]
        #Otherwise, load it then show it
        else:
            frame = f(self)
            self.frames[f] = frame
            frame.place(relx=0,rely=0,relwidth=1,relheight=1)
        frame.tkraise()

#=====PAGES=====

class HomePage(tk.Frame):
    def __init__(self,parent):
        tk.Frame.__init__(self, parent)

        self.label = ttk.Label(self, text="Welcome to AppName Grouping Tool!", font=LARGE_FONT)
        self.label.pack()

        self.projectButton = ttk.Button(self,text="Choose Projects",command=lambda:parent.show_frame(ProjectPage))
        self.projectButton.pack()

        self.studentButton = ttk.Button(self,text="Choose Students",command=lambda:parent.show_frame(StudentPage))
        self.studentButton.pack()

        self.runButton = ttk.Button(self,text="Create Groups",command=lambda:parent.show_frame(LoadingPage),state="disabled")
```

```
65          self.runButton.pack()
66
67          self.warning = ttk.Label(self,text="Must choose projects and
    ↪ students first")
68
69
70      #This method runs once everytime the page is opened
71      def tkraise(self):
72          tk.Frame.tkraise(self)
73
74          #If at least one student and one project has been selected, allow
    ↪  the run button to be clickable and remove the warning message
75          if ProjectPage in self.master.frames and StudentPage in self.
    ↪ master.frames and self.master.frames[ProjectPage].
    ↪ check_for_selection() and self.master.frames[StudentPage].
    ↪ check_for_selection():
76              self.runButton["state"]="enabled"
77              self.warning.pack_forget()
78
79          #Otherwise, disable the run button and display the warning
    ↪ message
80          else:
81              self.runButton["state"]="disabled"
82              self.warning.pack()
83
84
85  class ProjectPage(tk.Frame):
86      def __init__(self,parent):
87          tk.Frame.__init__(self,parent)
88
89          self.header=Header(self,"Projects",HEADER_COLOUR)
90          self.header.pack(fill="x")
91
92          self.backButton=ttk.Button(self.header,text="Back",style="Header.
    ↪ TButton",command=lambda:parent.show_frame(HomePage)) #Change to
    ↪ check where it should go back to
93          self.backButton.pack(side="right")
94
95          self.importButton=ttk.Button(self,text="Import from file",command
    ↪ =self.import_projects)
96          self.importButton.pack()
97
98          self.addButton=ttk.Button(self,text="New project",command=lambda:
    ↪ self.create_entry_window("New"))
99          self.addButton.pack()
100
101         #self.separator=ttk.Separator(self,orient="horizontal")
102         #self.separator.pack(fill="x",expand=True)
103
104         self.boxesFrame=ScrollableFrame(self)
```

```
105         self.boxesFrame.canvas.pack(fill="both",expand=True)
106         self.update()
107 
108         self.projectBoxes={}
109 
110         #Display projects stored in the database on the UI
111         projectquery="SELECT * FROM Projects"
112         skillquery="SELECT Skill FROM ProjectSkills WHERE ProjectID = ?"
113         #Fetch project data, except skills
114         with sql.connect(DATABASE) as conn:
115             cur = conn.cursor()
116             cur.execute(projectquery)
117             projects=cur.fetchall()
118 
119             for project in projects:
120                 ID = project[0]
121                 name=project[1]
122                 maxSize=project[2]
123                 minSize=project[3]
124                 optional=bool(project[4])
125 
126                 #Fetch each project's skill requirements
127                 cur.execute(skillquery,(ID,))
128                 dbskills=cur.fetchall()
129                 skills=[]
130                 for skill in dbskills:
131                     skills.append(skill[0])
132 
133                 #Display data on UI
134                 self.projectBoxes[ID]=projectBox(self.boxesFrame,self,ID,
    ↪ name,maxSize,minSize,optional,skills)
135                 self.projectBoxes[ID].pack()
136 
137 
138     #Method override as a work-around for a scrolling bug
139     def update(self):
140         self.master.state('normal')
141         tk.Frame.update(self)
142         self.master.state('zoomed')
143 
144     #Iterates through every project, and returns true if it finds a
    ↪ selected project. Returns false when no project is selected.
145     def check_for_selection(self):
146         for box in self.projectBoxes:
147             if self.projectBoxes[box].check_selection():
148                 return True
149         return False
150 
151     def import_projects(self):
```

```
152         #Show an "Open" dialog box and return an open file object in read
    ↪  mode
153         file = filedialog.askopenfile(mode='r',filetypes=[("Supported
    ↪ file types",".txt .csv")]) #Add excel
154         #Validation to ensure the user selected a file
155         if file == None:
156             return
157 
158         #<<<-------------------------------------------
159 
160         file.close()
161         return
162 
163     def create_entry_window(self,mode,**kwargs):
164         name=tk.StringVar()
165         maxSize=tk.StringVar()
166         minSize=tk.StringVar()
167         optional=tk.BooleanVar()
168         skills=tk.StringVar()
169 
170         if mode == "Edit":
171             popupWin=entryPopupWin(self,"Edit Project")
172             name.set(kwargs["name"])
173             maxSize.set(kwargs["maxSize"])
174             minSize.set(kwargs["minSize"])
175             optional.set(kwargs["optional"])
176             skills.set(", ".join(kwargs["skills"]))
177         else:
178             popupWin=entryPopupWin(self,"New project")
179 
180         popupWin.add_string_entry("Name:",name)
181 
182         popupWin.add_int_entry("Maximum size allowed for this project's
    ↪ group:",maxSize)
183 
184         popupWin.add_int_entry("Minimum size allowed for this project's
    ↪ group:",minSize)
185 
186         popupWin.add_checkbox("Optional:",optional)
187 
188         popupWin.add_string_entry("Skills, each one separated by a comma:
    ↪ ",skills)
189 
190         popupWin.add_save_cancel(popupWin,mode,ID=kwargs["ID"],name=name,
    ↪ maxSize=maxSize,minSize=minSize,optional=optional,skills=skills)
191 
192     def save_entry(self,window,mode,**kwargs):
193         name=kwargs["name"].get()
194         maxSize=kwargs["maxSize"].get()
195         minSize=kwargs["minSize"].get()
```

```
196             optional=kwargs["optional"].get()
197             skills=kwargs["skills"].get()
198 
199             #Validation to ensure all required entries are filled
200             if name != "" and maxSize != "" and minSize != "" and skills != "
    ↪ ":
201 
202                 maxSize = int(maxSize)
203                 minSize = int(minSize)
204 
205                 #Separate skills into a list and remove whitespace
206                 skills=skills.split(",")
207                 for i in range(len(skills)):
208                     skills[i]=skills[i].strip()
209 
210                 if mode == "Edit":
211                     self.edit_project(kwargs["ID"],name,maxSize,minSize,
    ↪ optional,skills)
212                 else:
213                     self.new_project(name,maxSize,minSize,optional,skills)
214 
215                 #Destroy the popup window
216                 window.destroy()
217 
218 
219     #Add the inputted data to the Project and Project Skills tables in
    ↪ the database
220     def new_project(self,name,maxSize,minSize,optional,skills):
221         mainquery="INSERT INTO Projects (Name, MaxSize, MinSize, Optional
    ↪ ) VALUES (?,?,?,?)"
222         skillsquery="INSERT INTO ProjectSkills (ProjectID, Skill) VALUES
    ↪ (?,?)"
223         with sql.connect(DATABASE) as conn:
224             cur = conn.cursor()
225 
226             cur.execute(mainquery,(name,maxSize,minSize,optional))
227 
228             #Fetch the autogenerated ID for this student
229             ID=cur.lastrowid
230 
231             for skill in skills:
232                 cur.execute(skillsquery,(ID, skill))
233 
234             conn.commit()
235 
236         #Display new project on the GUI
237         self.projectBoxes[ID]=projectBox(self.boxesFrame,self,ID,name,
    ↪ maxSize,minSize,optional,skills)
238         self.projectBoxes[ID].pack()
239 
```

```
240         #Work-around for a scrollbar bug
241         self.update()
242 
243     def edit_project(self,ID,name,maxSize,minSize,optional,skills):
244         edited=False
245         box=self.projectBoxes[ID]
246         oldData=box.get_data()
247 
248         #Only updates if the new data is different from the old data
249         with sql.connect(DATABASE) as conn:
250             cur = conn.cursor()
251 
252             if name != oldData["name"]:
253                 edited=True
254 
255                 query = "UPDATE Projects SET Name = ? WHERE ProjectID = ?
    ↪ "
256                 cur.execute(query,(name,ID))
257 
258                 box.edit_data(name=name)
259 
260             if maxSize != oldData["maxSize"]:
261                 edited=True
262 
263                 query="UPDATE Projects SET maxSize = ? WHERE ProjectID =
    ↪ ?"
264                 cur.execute(query,(maxSize,ID))
265 
266                 box.edit_data(maxSize=maxSize)
267 
268             if minSize != oldData["minSize"]:
269                 edited=True
270 
271                 query="UPDATE Projects SET minSize = ? WHERE ProjectID =
    ↪ ?"
272                 cur.execute(query,(minSize,ID))
273 
274                 box.edit_data(minSize=minSize)
275 
276             if optional != oldData["optional"]:
277                 edited=True
278 
279                 query="UPDATE Projects SET optional = ? WHERE ProjectID =
    ↪  ?"
280                 cur.execute(query,(optional,ID))
281 
282                 box.edit_data(optional=optional)
283 
284             if skills != oldData["skills"]:
285                 edited=True
```

```
286
287                 cur.execute("DELETE FROM ProjectSkills WHERE ProjectID =
    ↪ ?",(ID,))
288                 query = "INSERT INTO ProjectSkills (ProjectID, Skill)
    ↪ VALUES (?,?)"
289                 for skill in skills:
290                     cur.execute(query,(ID,skill))
291
292                 box.edit_data(skills=skills)
293
294             if edited:
295                 self.update()
296                 conn.commit()
297
298     def editBox(self,boxID):
299         box=self.projectBoxes[boxID]
300         data=box.get_data()
301         self.create_entry_window("Edit",ID=data["ID"],name=data["name"],
    ↪ maxSize=data["maxSize"],minSize=data["minSize"], optional=data["
    ↪ optional"], skills=data["skills"])
302
303     def deleteBox(self,boxID):
304         #Delete from database
305         query1 = "DELETE FROM Projects WHERE ProjectID=?"
306         query2 = "DELETE FROM ProjectSkills WHERE ProjectID=?"
307         with sql.connect(DATABASE) as conn:
308             cur = conn.cursor()
309             cur.execute(query1,(boxID,))
310             cur.execute(query2,(boxID,))
311             conn.commit()
312         #Delete from UI
313         self.projectBoxes[boxID].pack_forget()
314         #Delete from internal store
315         del self.projectBoxes[boxID]
316
317         self.update()
318
319
320
321 class StudentPage(tk.Frame):
322     def __init__(self,parent):
323         tk.Frame.__init__(self, parent)
324
325         self.header=Header(self,"Students",HEADER_COLOUR)
326         self.header.pack(fill="x")
327
328         self.backButton=ttk.Button(self.header,text="Back",style="Header.
    ↪ TButton",command=lambda:parent.show_frame(HomePage)) #Change to
    ↪ check where it should go back to
329         self.backButton.pack(side="right")
```

```
330
331         self.importButton=ttk.Button(self,text="Import from file",command
    ↪ =self.import_students)
332         self.importButton.pack()
333
334         self.addButton=ttk.Button(self,text="New student",command=lambda:
    ↪ self.create_entry_window("New"))
335         self.addButton.pack()
336
337         self.boxesFrame=ScrollableFrame(self)
338         self.boxesFrame.canvas.pack(fill="both",expand=True)
339         self.update()
340
341         self.studentBoxes={}
342
343         #Display students stored in the database on the UI
344         studentquery="SELECT * FROM Students"
345         skillquery="SELECT Skill FROM StudentSkills WHERE StudentID=?"
346         #Fetch student data, except skills
347         with sql.connect(DATABASE) as conn:
348             cur = conn.cursor()
349             cur.execute(studentquery)
350             students=cur.fetchall()
351
352             for student in students:
353                 ID=student[0]
354                 name=student[1]
355                 gender=student[2]
356
357                 #Fetch each student's skills
358                 cur.execute(skillquery,(ID,))
359                 dbskills=cur.fetchall()
360                 skills=[]
361                 for skill in dbskills:
362                     skills.append(skill[0])
363
364                 #Display data on UI
365                 self.studentBoxes[ID]=studentBox(self.boxesFrame,self,ID,
    ↪ name,gender,skills)
366                 self.studentBoxes[ID].pack()
367
368     #Method override as a work-around for a scrolling bug
369     def update(self):
370         self.master.state('normal')
371         tk.Frame.update(self)
372         self.master.state('zoomed')
373
374     def check_for_selection(self):
375         for box in self.studentBoxes:
376             if self.studentBoxes[box].check_selection():
```

```
377                 return True
378         return False
379 
380     def import_students(self):
381         #Show an "Open" dialog box and return an open file object in read
    ↪  mode
382         file = filedialog.askopenfile(mode='r',filetypes=[("Supported
    ↪ file types",".txt .csv")]) #Add excel
383         #Validation to ensure the user selected a file
384         if file == None:
385             return
386 
387         #<<<-----------------------------------------
388 
389         file.close()
390         return
391 
392     def create_entry_window(self,mode,**kwargs):
393         name=tk.StringVar()
394         gender=tk.StringVar()
395         skills=tk.StringVar()
396 
397         if mode == "Edit":
398             popupWin=entryPopupWin(self, "Edit student")
399             name.set(kwargs["name"])
400             gender.set(kwargs["gender"])
401             skills.set(", ".join(kwargs["skills"]))
402         else:
403             popupWin=entryPopupWin(self,"New student")
404 
405         popupWin.add_string_entry("Name:", name)
406 
407         popupWin.add_string_entry("Gender:",gender)
408 
409         popupWin.add_string_entry("Skills, each one separated by a comma:
    ↪ ",skills)
410 
411         popupWin.add_save_cancel(popupWin,mode,ID=kwargs["ID"],name=name,
    ↪ gender=gender,skills=skills)
412 
413     def save_entry(self,window,mode,**kwargs):
414         name=kwargs["name"].get()
415         gender=kwargs["gender"].get()
416         skills=kwargs["skills"].get()
417 
418         #Validation to ensure all required entries are filled
419         if name != "" and gender != "" and skills != "":
420 
421             #Separate skills into a list and remove whitespace
422             skills=skills.split(",")
```

```
423             for i in range(len(skills)):
424                 skills[i]=skills[i].strip()
425 
426             if mode == "Edit":
427                 self.edit_student(kwargs["ID"],name,gender,skills)
428             else:
429                 self.new_student(name,gender,skills)
430 
431             #Destroy the popup window
432             window.destroy()
433 
434 
435     #Add the inputted data to the Student and StudentSkills tables in the
    ↪  database
436     def new_student(self,name,gender,skills):
437         mainquery="INSERT INTO Students (Name, Gender) VALUES (?,?)"
438         skillsquery="INSERT INTO StudentSkills (StudentID, Skill) VALUES
    ↪ (?,?)"
439         with sql.connect(DATABASE) as conn:
440             cur = conn.cursor()
441 
442             cur.execute(mainquery,(name,gender))
443 
444             #Fetch the autogenerated ID for this student
445             ID=cur.lastrowid
446 
447             for skill in skills:
448                 cur.execute(skillsquery,(ID, skill))
449 
450             conn.commit()
451 
452         #Display new student on the GUI
453         self.studentBoxes[ID]=studentBox(self.boxesFrame,self,ID,name,
    ↪ gender,skills)
454         self.studentBoxes[ID].pack()
455 
456         #Scroll bar bug work-around
457         self.update()
458 
459 
460     def edit_student(self,ID,name,gender,skills):
461         edited=False
462 
463         box=self.studentBoxes[ID]
464         oldData=box.get_data()
465 
466         #Only updates if the new data is different from the old data
467         with sql.connect(DATABASE) as conn:
468             cur = conn.cursor()
469 
```

```
470             if name != oldData["name"]:
471                 edited=True
472 
473                 query = "UPDATE Students SET Name = ? WHERE StudentID = ?
    ↪ "
474                 cur.execute(query,(name,ID))
475 
476                 box.edit_data(name=name)
477 
478             if gender != oldData["gender"]:
479                 edited=True
480 
481                 query="UPDATE Students SET Gender = ? WHERE StudentID = ?
    ↪ "
482                 cur.execute(query,(gender,ID))
483 
484                 box.edit_data(gender=gender)
485 
486             if skills != oldData["skills"]:
487                 edited=True
488 
489                 cur.execute("DELETE FROM StudentSkills WHERE StudentID =
    ↪ ?",(ID,))
490                 query = "INSERT INTO StudentSkills (StudentID, Skill)
    ↪ VALUES (?,?)"
491                 for skill in skills:
492                     cur.execute(query,(ID,skill))
493 
494                 box.edit_data(skills=skills)
495 
496             if edited:
497                 self.update()
498                 conn.commit()
499 
500     def editBox(self,boxID):
501         box=self.studentBoxes[boxID]
502         data=box.get_data()
503         self.create_entry_window("Edit", ID=data["ID"],name=data["name"],
    ↪ gender=data["gender"],skills=data["skills"])
504 
505     def deleteBox(self,boxID):
506         #Delete from database
507         query1 = "DELETE FROM Students WHERE StudentID=?"
508         query2 = "DELETE FROM StudentSkills WHERE StudentID=?"
509         with sql.connect(DATABASE) as conn:
510             cur = conn.cursor()
511             cur.execute(query1,(boxID,))
512             cur.execute(query2,(boxID,))
513             conn.commit()
514         #Delete from UI
```

```
515         self.studentBoxes[boxID].pack_forget()
516         #Delete from internal store
517         del self.studentBoxes[boxID]
518 
519         self.update()
520 
521 
522 
523 class LoadingPage(tk.Frame):
524     def __init__(self,parent):
525         tk.Frame.__init__(self, parent)
526 
527         self.label = ttk.Label(self, text="Grouping in progress...")
528         self.label.pack()
529 
530         self.cancelButton=ttk.Button(self,text="Cancel",command=lambda:
    ↪ parent.show_frame(HomePage))
531         self.cancelButton.pack()
532 
533 class ResultsPage(tk.Frame):
534     def __init__(self,parent):
535         tk.Frame.__init__(self, parent)
536 
537         self.header=Header(self,"Results",HEADER_COLOUR)
538         self.header.pack(fill="x")
539 
540         self.homeButton=ttk.Button(self.header,text="Back",style="Header.
    ↪ TButton",command=lambda:parent.show_frame(HomePage))
541         self.homeButton.pack(side="right")
542 
543 class RegisterPage(tk.Frame):
544     def __init__(self,parent):
545         tk.Frame.__init__(self, parent)
546 
547         self.header=Header(self,"Register",HEADER_COLOUR)
548         self.header.pack(fill="x")
549 
550         self.homeButton=ttk.Button(self.header,text="Back",style="Header.
    ↪ TButton",command=lambda:parent.show_frame(HomePage))
551         self.homeButton.pack(side="right")
552 
553 #=====WIDGETS=====
554 
555 class Header(tk.Frame):
556     def __init__(self,parent,text,colour):
557         tk.Frame.__init__(self,parent)
558         self.config(bg=colour)
559         self.label = tk.Label(self, text=text, bg=colour,font=LARGE_FONT)
    ↪  #change to ttk
560         self.label.pack(side="left")
```

```
561
562 class ScrollableFrame(tk.Frame):
563     def __init__(self,parent):
564         self.canvas=tk.Canvas(parent,background="white")
565         tk.Frame.__init__(self,self.canvas)
566
567         self.scrollbar=tk.Scrollbar(self.canvas,orient="vertical",command
    ↪ =self.canvas.yview)
568
569         self.canvas.configure(yscrollcommand=self.scrollbar.set)
570
571         self.scrollbar.pack(side="right",fill="y")
572         self.scrollbar.update()
573
574         self.canvas.create_window((0,0),window=self,anchor='nw',width=
    ↪ self.winfo_screenwidth()-self.scrollbar.winfo_width())
575         self.canvas.bind("<Configure>",self.scroll)
576         #self.canvas.bind_all("<MouseWheel>", self._on_mousewheel)
577     #def _on_mousewheel(self, event):
578         #self.canvas.yview_scroll(int(-1*(event.delta/120)), "units")
579     def scroll(self,event):
580         self.canvas.configure(scrollregion=self.canvas.bbox("all"),width=
    ↪ self.winfo_width(),height=self.winfo_height())
581
582 #Has checkbox, name, edit button, delete button.
583 class selectionBox(tk.Frame):
584     def __init__(self,parent,page,ID,name):
585         tk.Frame.__init__(self,parent,relief="solid",bd=2)
586
587         self.ID=ID
588         self.name=name
589         self.page=page
590
591         self.selected=tk.BooleanVar(value=False)
592         self.checkbox=ttk.Checkbutton(self,variable=self.selected,onvalue
    ↪ =True,offvalue=False)
593         self.checkbox.pack()
594
595         self.nameLabel=ttk.Label(self,text=name)
596         self.nameLabel.pack()
597
598         self.editButton=ttk.Button(self,text="Edit",command=self.edit)
599         self.editButton.pack()
600
601         self.deleteButton=ttk.Button(self,text="Delete",command=self.
    ↪ delete)
602         self.deleteButton.pack()
603
604     def check_selection(self):
605         return self.selected.get()
```

```
606
607     def edit(self):
608         self.page.editBox(self.ID)
609
610     def delete(self):
611         delChoice=messagebox.askyesno(title="Delete entry", message=f"Are
    ↪  you sure you want to delete {self.name}?")
612         if delChoice:
613             self.selected.set(False)
614             self.page.deleteBox(self.ID)
615
616     def pack(self):
617         tk.Frame.pack(self,fill="x",padx=10,pady=5)
618
619     #-----Abstract methods-----
620     def get_data(self):
621         return
622
623     def edit_data(self):
624         return
625
626
627 class projectBox(selectionBox):
628     def __init__(self,parent,page,ID,name,maxSize,minSize,optional,skills
    ↪ ):
629         selectionBox.__init__(self,parent,page,ID,name)
630         self.maxSize=maxSize
631         self.minSize=minSize
632         self.optional=optional
633         self.skills=skills
634
635         self.maxSizeLabel=ttk.Label(self,text=f"Max group size: {maxSize}
    ↪ ")
636         self.maxSizeLabel.pack()
637
638         self.minSizeLabel=ttk.Label(self,text=f"Min group size: {minSize}
    ↪ ")
639         self.minSizeLabel.pack()
640
641         self.optionalLabel=ttk.Label(self,text=f"Optional: {optional}")
642         self.optionalLabel.pack()
643
644         self.skillsLabel=ttk.Label(self,text=f"Skills: {", ".join(skills)
    ↪ }")
645         self.skillsLabel.pack()
646
647     def get_data(self):
648         return {"ID":self.ID,"name":self.name,"maxSize":self.maxSize,"
    ↪ minSize":self.minSize,"optional":self.optional,"skills":self.
    ↪ skills}
```

```
649
650 class studentBox(selectionBox):
651     def __init__(self, parent, page, ID, name, gender, skills):
652         selectionBox.__init__(self,parent,page,ID,name)
653         self.gender=gender
654         self.skills=skills
655
656         self.genderLabel = ttk.Label(self,text=f"Gender: {gender}")
657         self.genderLabel.pack()
658
659         self.skillsLabel = ttk.Label(self, text= f"Skills: {", ".join(
    ↪ skills)}")
660         self.skillsLabel.pack()
661
662     def get_data(self):
663         return {"ID":self.ID,"name":self.name,"gender":self.gender,"
    ↪ skills":self.skills}
664
665     def edit_data(self,**kwargs):
666         if "name" in kwargs:
667             self.name=kwargs["name"]
668             self.nameLabel.config(text=self.name)
669         if "gender" in kwargs:
670             self.gender=kwargs["gender"]
671             self.genderLabel.config(text=f"Gender: {self.gender}")
672         if "skills" in kwargs:
673             self.skills=kwargs["skills"]
674             self.skillsLabel.config(text=f"Skills: {", ".join(self.skills
    ↪ )}")
675
676
677 class entryPopupWin(tk.Toplevel):
678     def __init__(self,parent,title):
679         tk.Toplevel.__init__(self,parent)
680         self.title(title)
681
682     def add_string_entry(self,text,variable):
683         ttk.Label(self, text=text).pack()
684         ttk.Entry(self, textvariable=variable).pack()
685
686     def add_int_entry(self,text,variable,minimum=0,maximum=100,increment=
    ↪ 1):
687
688         #Prevents user from typing non numbers into number-only entry
    ↪ boxes
689         def intCallback(P):
690             return str.isdigit(P) or P == ""
691
692         intValidate = (self.register(intCallback), '%P')
693
```

```
694         ttk.Label(self, text=text).pack()
695         ttk.Spinbox(self, textvariable=variable,validate="all",
    ↪ validatecommand=intValidate,from_=minimum,increment=increment,to=
    ↪ maximum).pack()
696 
697     def add_checkbox(self,text,variable):
698         ttk.Label(self, text=text).pack()
699         ttk.Checkbutton(self, variable=variable, onvalue=True, offvalue=
    ↪ False).pack()
700 
701     def add_save_cancel(self,window,mode,**kwargs):
702         ttk.Button(self, text="Save", command=lambda:self.master.
    ↪ save_entry(window,mode,**kwargs)).pack()
703         ttk.Button(self, text="Cancel", command=self.destroy).pack()
704 
705 #=====MAIN=====
706 
707 app=Window()
708 app.mainloop()
```

3.2 algoClasses.py

```
1  class Project:
2      def __init__(self,name,skills,maxSize,minSize,optional,group=None):
3          self.name=name
4          self.skills=skills
5          self.maxSize=maxSize
6          self.minSize=minSize
7          self.optional=optional
8  
9          self.group=group
10         self.fitness=self.calcFitness()
11 
12     def calcFitness():
13         return
14 
15 class Student:
16     def __init__(self,name,skills,gender):
17         self.name=name
18         self.skills=skills
19         self.gender=gender
20 
21 class Individual:
22     def __init__(self,projects):
23         self.projects=projects
24         self.fitness=self.calcFitness()
25     def calcFitness(self):
```

```
26          return
```

3.3 geneticAlgo.py

```
1  from algoClasses import Project,Student,Individual
2  import sqlite3 as sql
3
4  def get_students(database,IDs):
5      students=[]
6
7      with sql.connect(database) as conn:
8              cur = conn.cursor()
9
10
11             for ID in IDs:
12                 #Get each students' skills
13                 cur.execute("SELECT Skill FROM StudentSkills WHERE
       ↪ StudentID = ?",(ID))
14                 skills=cur.fetchall()
15
16                 #Get each students' name and gender
17                 cur.execute("SELECT Name, Gender from Students WHERE
       ↪ StudentID = ?",(ID))
18                 entry=cur.fetchall()
19
20                 #Create a Student object for each student with their name
       ↪ , skills, and gender
21                 students.append(Student(entry[0],skills,entry[1]))
22
23     return students
24
25
26 def get_projects(database,IDs):
27     projects=[]
28
29     with sql.connect(database) as conn:
30             cur = conn.cursor()
31
32             for ID in IDs:
33                 #Get each projects' skill requirements
34                 cur.execute("SELECT Skill FROM ProjectSkills WHERE
       ↪ ProjectID = ?",(ID))
35                 skills=cur.fetchall()
36
37                 #Get each projects' data other than skills
38                 cur.execute("SELECT Name, MaxSize, MinSize, Optional from
       ↪  Projects WHERE ProjectID = ?",(ID))
```

```
                entry=cur.fetchall()

                #Create a Project object for each project with its name, skills, maxSize, minSize, and whether or not it is optional
                projects.append(Project(entry[0],skills,entry[1],entry[2],entry[3]))

    return projects



def run(database, studentIDs, projectIDs, target_fitness, max_iterations, population_size):
    students=get_students(database,studentIDs)
    projects=get_projects(database,projectIDs)

    #Ensure there aren't too many or too few students to meet requirements
```

4 Testing

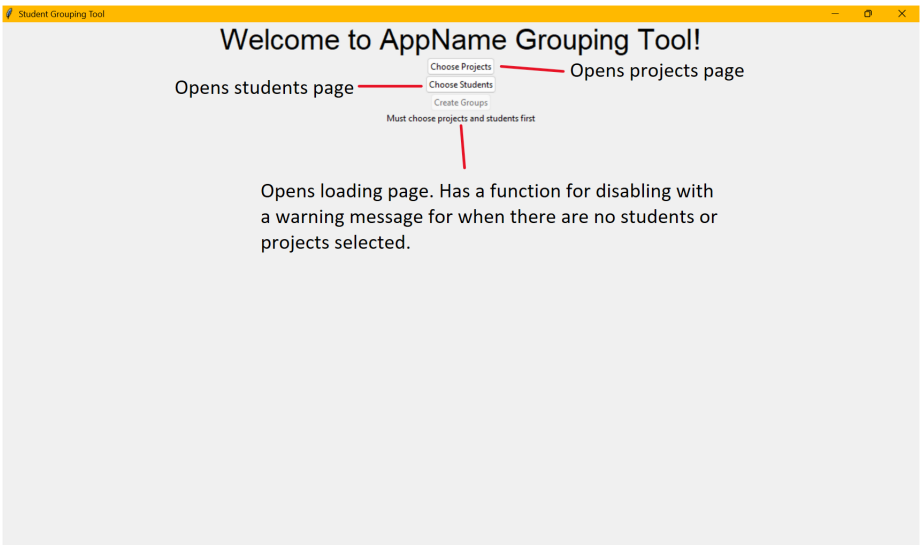


Figure 19: The home page

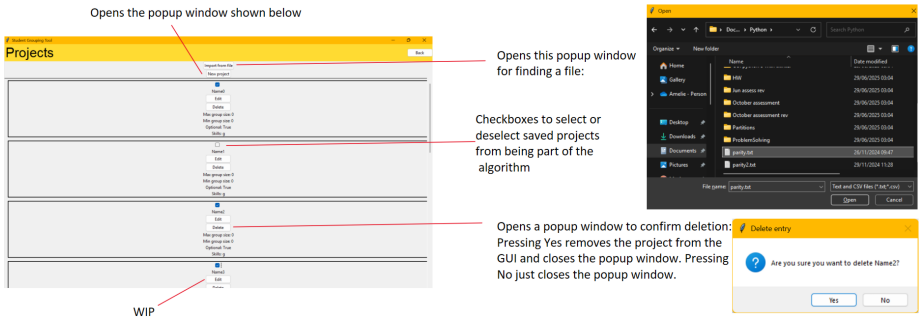


Figure 20: The project page

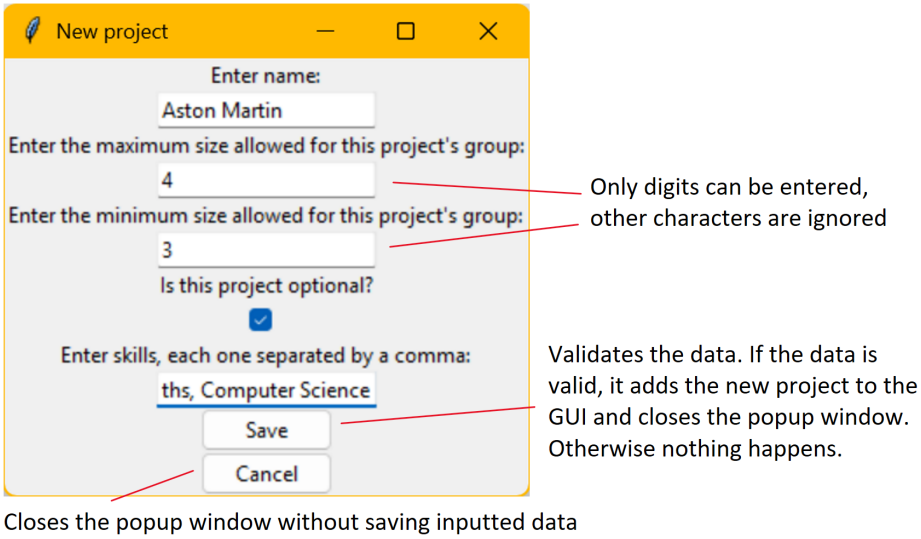


Figure 21: The popup window for entering data about a new project

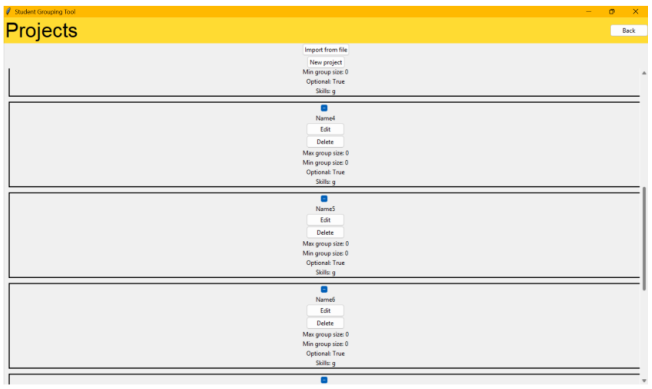


Figure 22: The project page, scrolled down

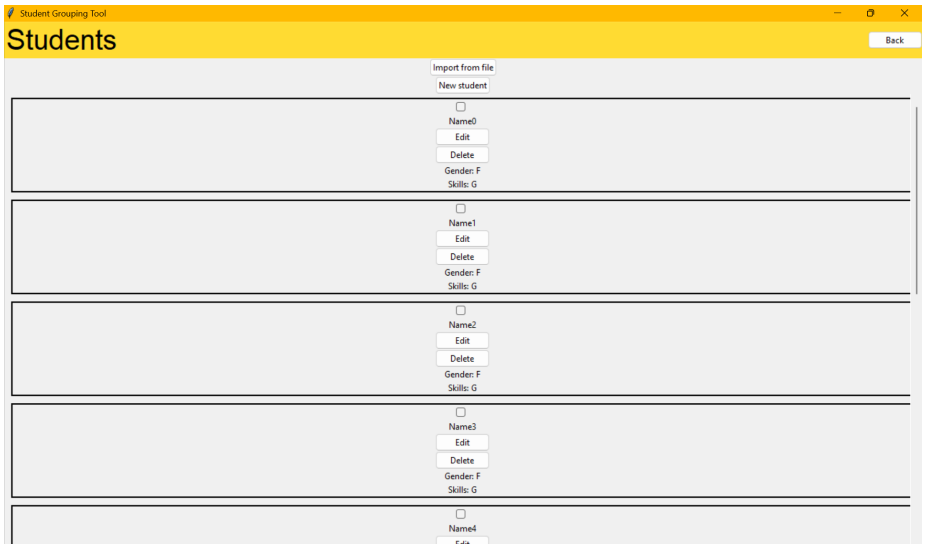


Figure 23: The student page. Functions identically to the projects page

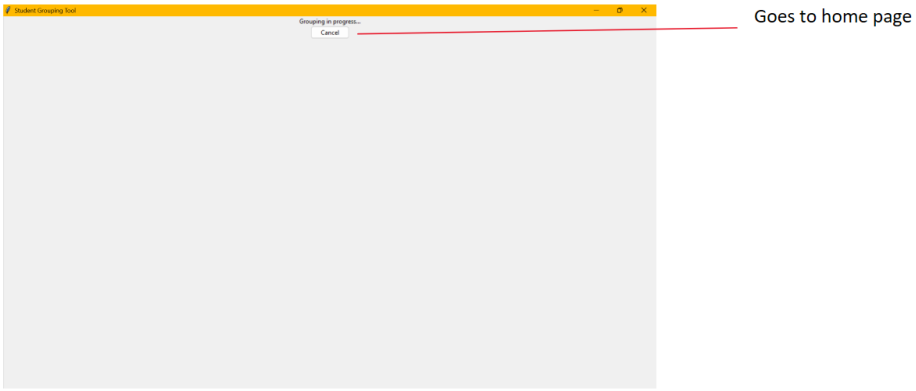


Figure 24: The loading page

5 Evaluation

So far, I have created a basic GUI with minimal formatting, and some basic functionality such as page navigation, scrollable frames, project management and student management.

I have also created the classes required by the genetic algorithm and made a start coding up my pseudocode for the genetic algorithm.

The fitness algorithm is harder to design than I thought it would be. I got a bit carried away working on the GUI and didn't spend as much time as I needed on the fitness algorithm so I haven't got one yet. I also didn't add much to the design section because of this, so I will add more diagrams to the design section before the next submission.